

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 6

End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the (N, T, F) input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

3. Dataset and Experimental Setup

Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING_UPSTAIRS
- WALKING_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

Recommended laboratory subset: To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

Important: The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

Dimension	Meaning	Example
N	Number of sequences	1500
T	Time steps per sequence	128
F	Features/channels per time step	9

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as 128×9 .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

Suggested split:

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

Expected Preprocessing Output

Students must print:

Input tensor shape:

```
Training   : (N_train, 128, 9)
Validation : (N_val,   128, 9)
Testing    : (N_test,  128, 9)
```

Number of classes: 6
Number of features per time step: 9
Sequence length: 128

The actual values of N_{train} , N_{val} and N_{test} depend on the selected subset.

6. Temporal Data Visualization

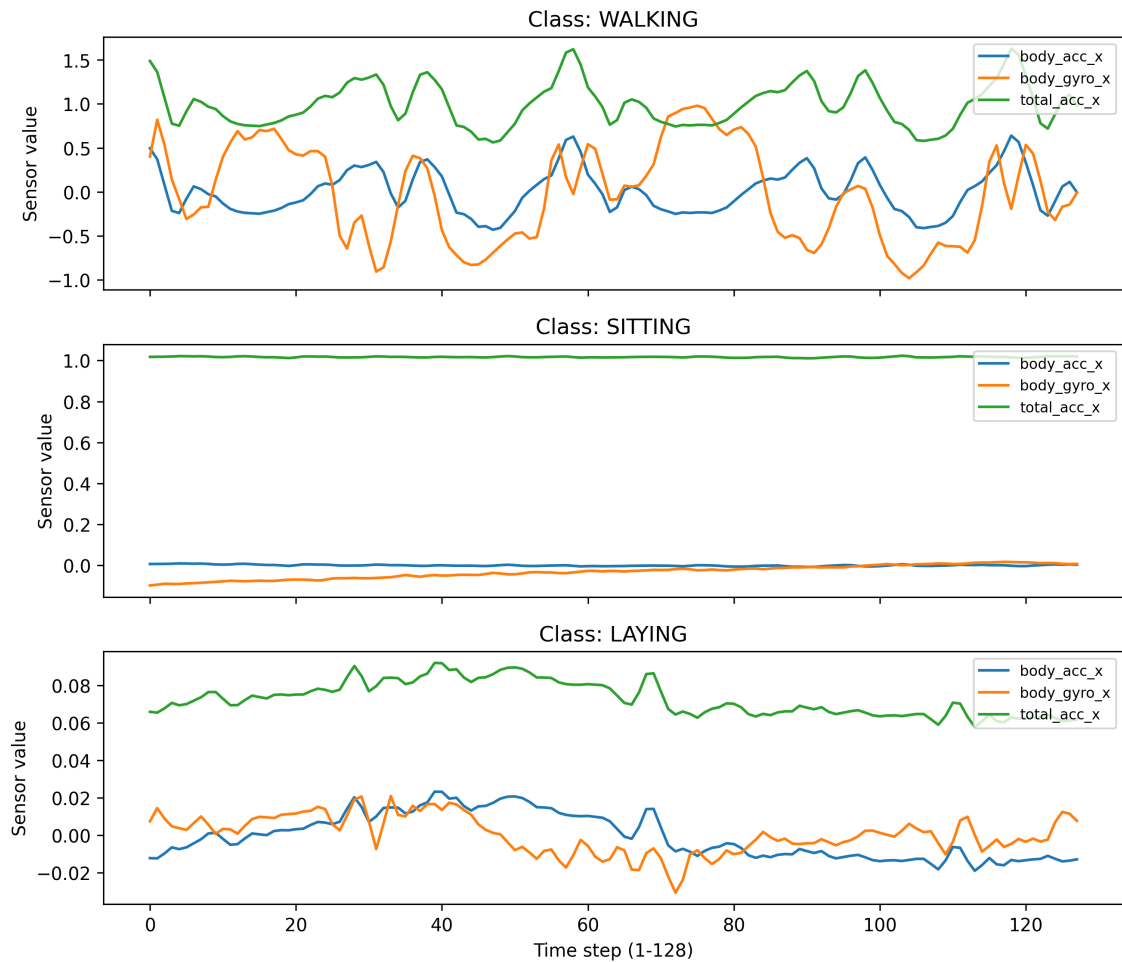
Select representative sequences from at least three different activity classes.

Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

Students should observe that temporal patterns differ across activities.

Plot 1: Sensor signal vs. time across activity classes



Required inference:

1. What temporal pattern is visible?
2. Which activities exhibit similar patterns?
3. Why is the ordering of measurements important?

Answer: WALKING oscillates continuously across all 128 time steps with amplitude close to ± 1 , while SITTING and LAYING stay nearly flat, with `total_acc` pinned near 1.0 from gravity. SITTING and STANDING look the most alike of the six classes in the raw signal, which lines up with the confusion they cause later in Section 15. Order matters because the model has to learn the shape of a motion over time, and the individual readings alone do not capture that – shuffling the 128 steps would destroy the shape even though the readings stay the same.

7. Vanilla RNN Architecture

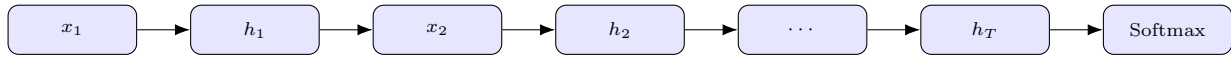
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

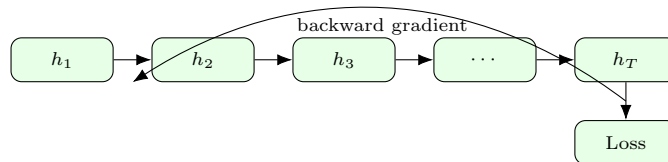
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate h_1 , h_2 and h_3 manually.

Students must compare their calculated values with the values obtained from a program.

Manual calculation:

$$h_1 = \tanh(0.5 \times 0.5 + 0.8 \times 0 + 0.1) = \tanh(0.35) = 0.336376$$

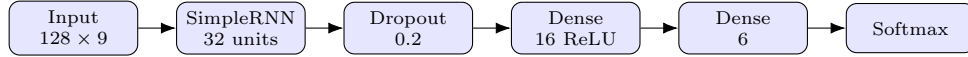
$$h_2 = \tanh(0.5 \times 0.7 + 0.8 \times 0.336376 + 0.1) = \tanh(0.719101) = 0.616352$$

$$h_3 = \tanh(0.5 \times 0.2 + 0.8 \times 0.616352 + 0.1) = \tanh(0.693082) = 0.599958$$

These values match the output of the program exactly (to the printed precision), confirming that the recurrence $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ was implemented correctly.

9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

Parameter	Value
Recurrent units	32
Dropout	0.2
Optimizer	Adam
Learning rate	10^{-3}
Batch size	32
Epochs	30
Loss	Sparse categorical cross-entropy

10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

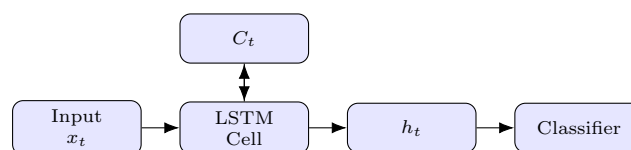
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



Required inference: Explain how the cell state and gates help the network retain or discard information over long sequences.

11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

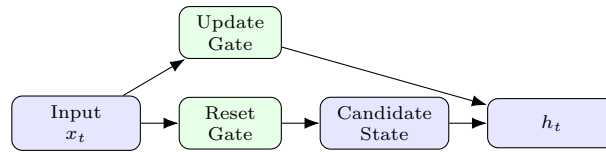
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

Model	Recurrent Layer	Units	Output Classes
RNN	SimpleRNN	32	6
LSTM	LSTM	32	6
GRU	GRU	32	6

All other experimental conditions should remain unchanged.

13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

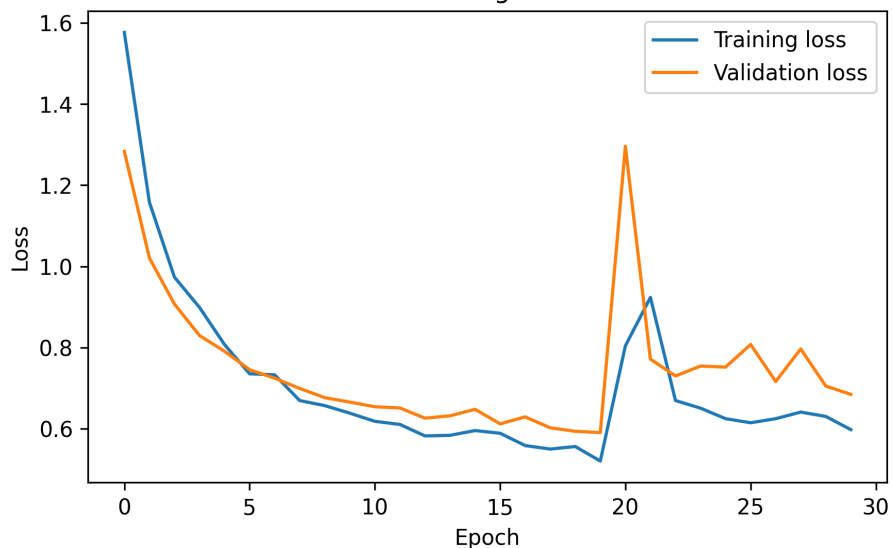
Plot 2: Training and Validation Loss

- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

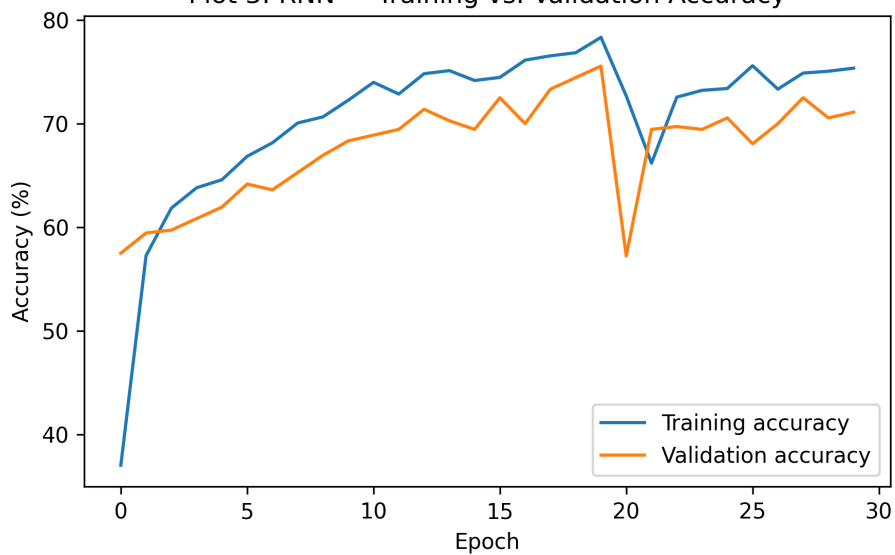
Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

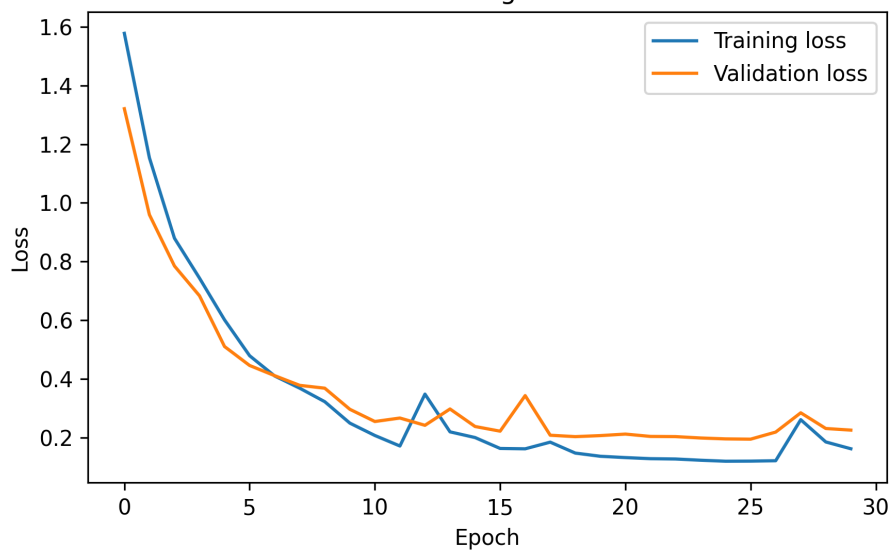
Plot 2: RNN — Training vs. Validation Loss

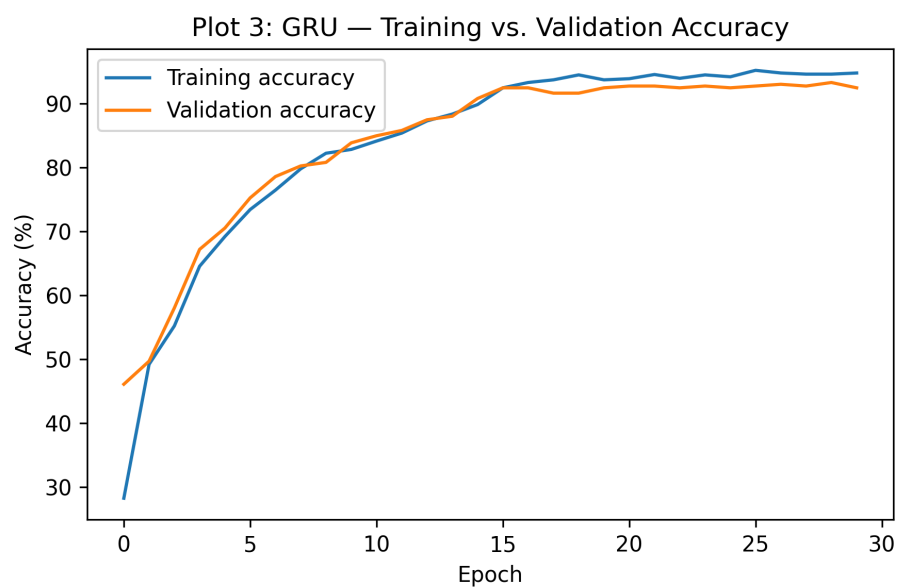
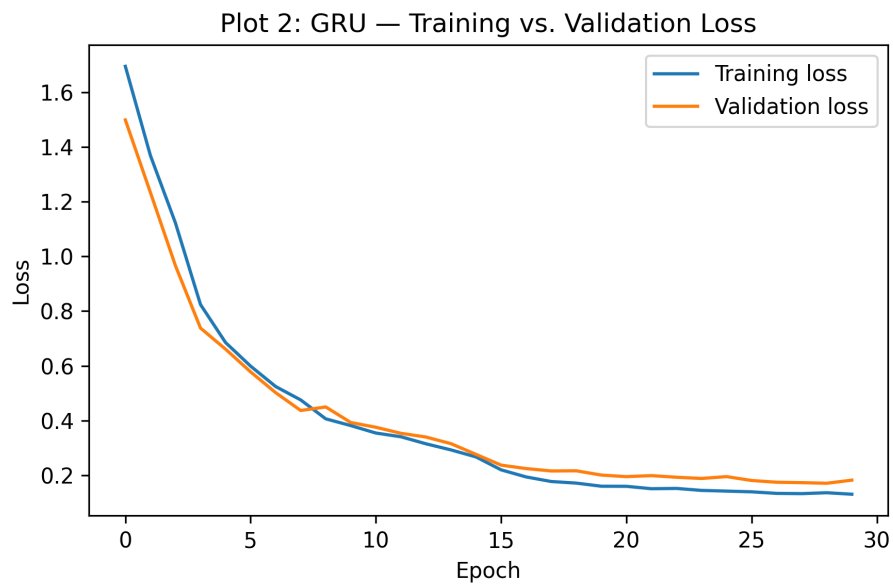
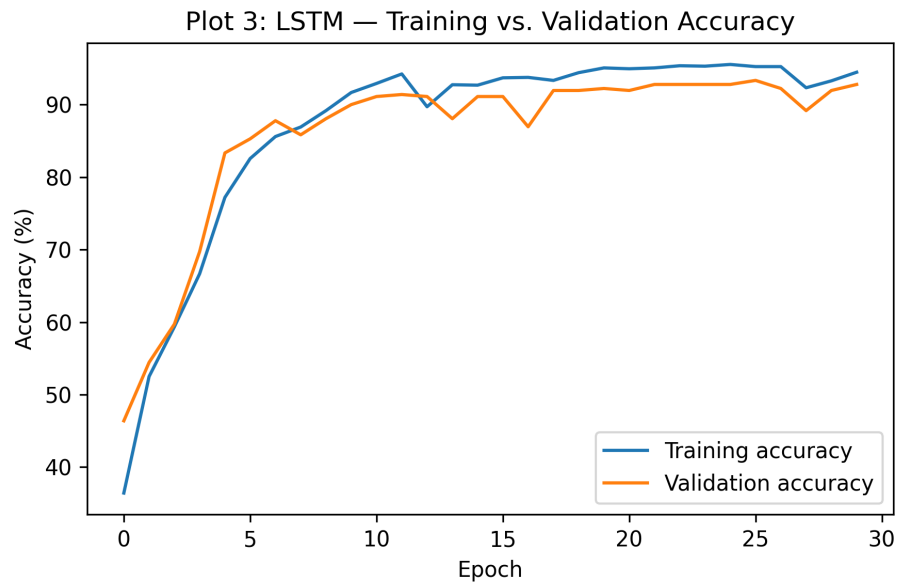


Plot 3: RNN — Training vs. Validation Accuracy



Plot 2: LSTM — Training vs. Validation Loss





Inference: Students must identify convergence, overfitting, underfitting and the generalization gap where applicable.

Answer – RNN: Training and validation loss move together for about 19 epochs, then both spike sharply near epoch 20 – validation loss jumps from roughly 0.59 to 1.29 and accuracy drops from 76% to 57%. The curves partly recover but stay noisy for the rest of training, matching the gradient instability Vanilla RNNs are prone to (Section

8). The RNN ends training with a higher loss than either LSTM or GRU.

Answer – LSTM: Loss drops quickly over the first 8 epochs and then decreases smoothly, with only small, short-lived validation spikes at epochs 12, 16 and 27 that recover within an epoch. Training and validation curves stay close together throughout, leaving a small, stable gap by epoch 30. This points to good convergence with at most mild overfitting.

Answer – GRU: Training and validation loss decrease almost identically for all 30 epochs, with no visible spikes – the smoothest run of the three. The train/val gap is the smallest of all three models by the end of training. This fits GRU’s simpler gating, which appears to make optimization more stable on this dataset.

14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

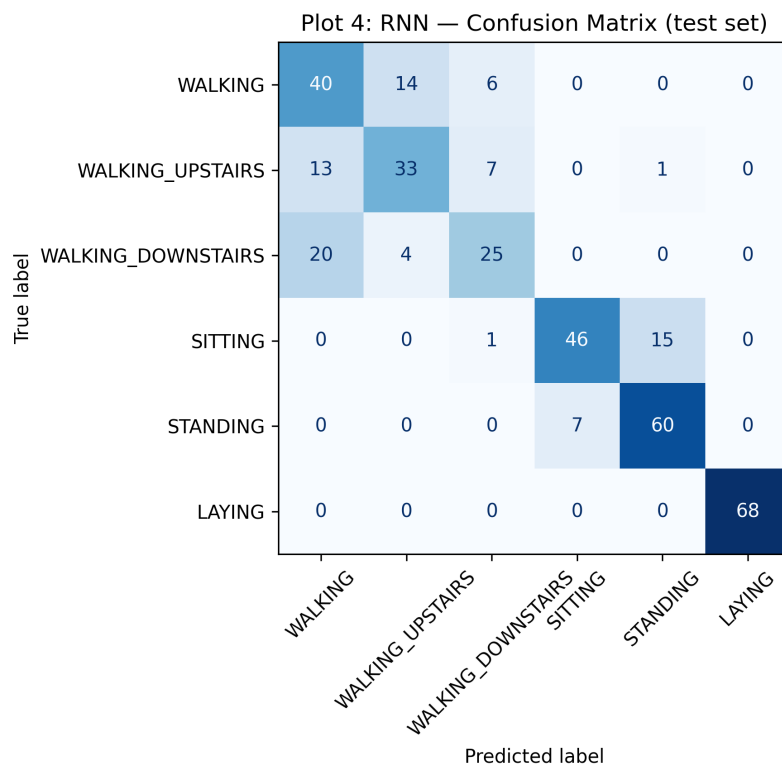
For multi-class classification, report macro-averaged Precision, Recall and F1-score.

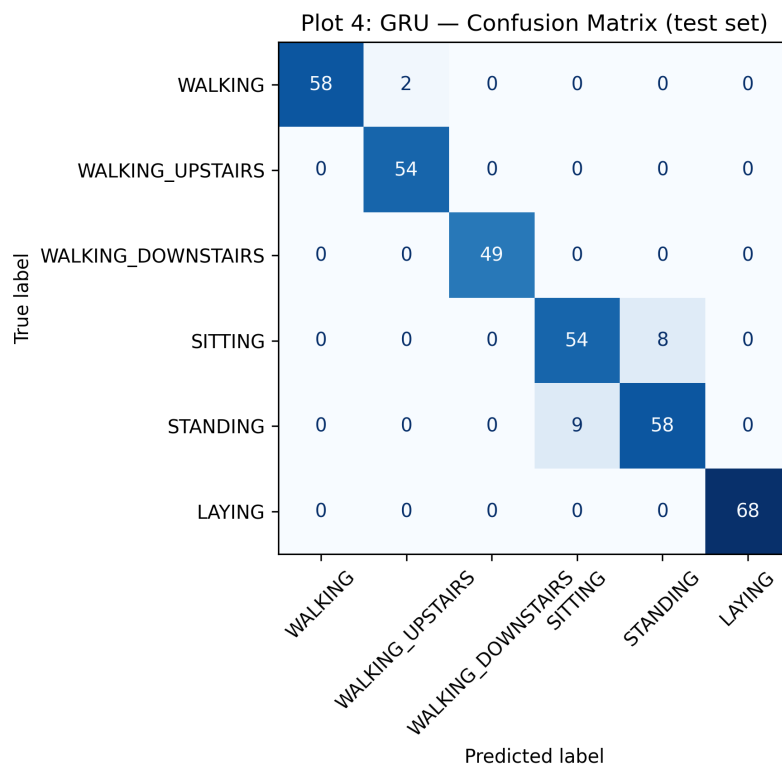
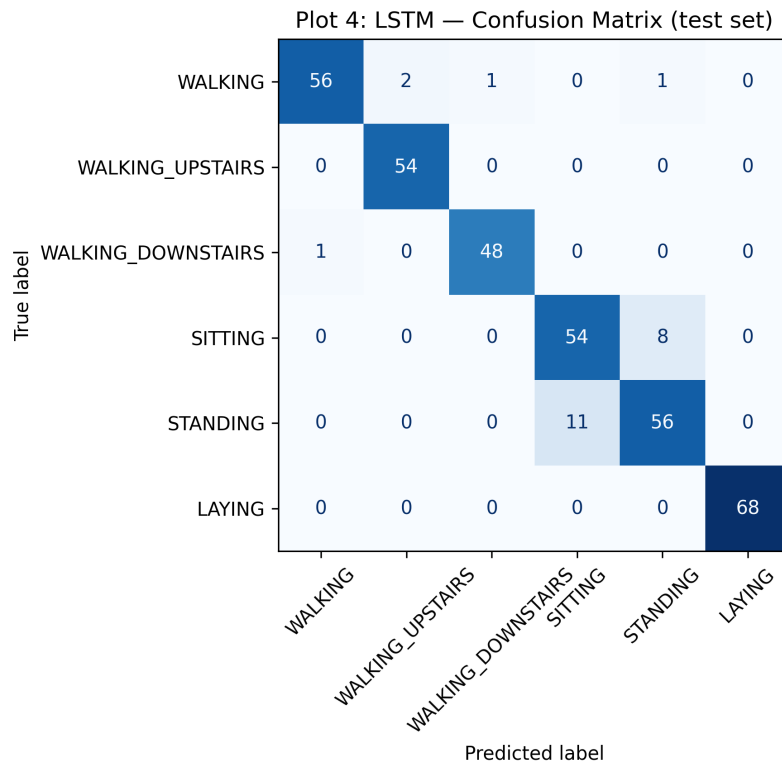
Metric	RNN	LSTM	GRU
Accuracy (%)	75.56	93.33	94.72
Macro Precision (%)	74.89	93.64	95.00
Macro Recall (%)	73.76	93.66	95.06
Macro F1 (%)	73.96	93.63	95.02
Parameters	1974	6006	4758
Training Time (s)	24.69	22.01	19.08

15. Confusion Matrix Analysis

Plot 4: Confusion Matrix

Generate a separate confusion matrix for each model.





Students must identify:

- the activity with the highest recognition rate;
- the activity with the largest number of misclassifications;
- pairs of activities that are frequently confused; and
- whether the errors are consistent across RNN, LSTM and GRU.

The inference must be based on the student's observed matrix rather than on an assumed result.

Answer: LAYING is classified perfectly by all three models (68/68), making it the most separable class. The worst-recognized class differs by model – WALKING_DOWNSTAIRS for RNN (25/49, confused with the other walking types) versus STANDING for LSTM and GRU (56/67 and 58/67, confused mainly with SITTING). SITTING and STANDING are confused in every model, while only RNN also mixes up the three walking-type activities with each other.

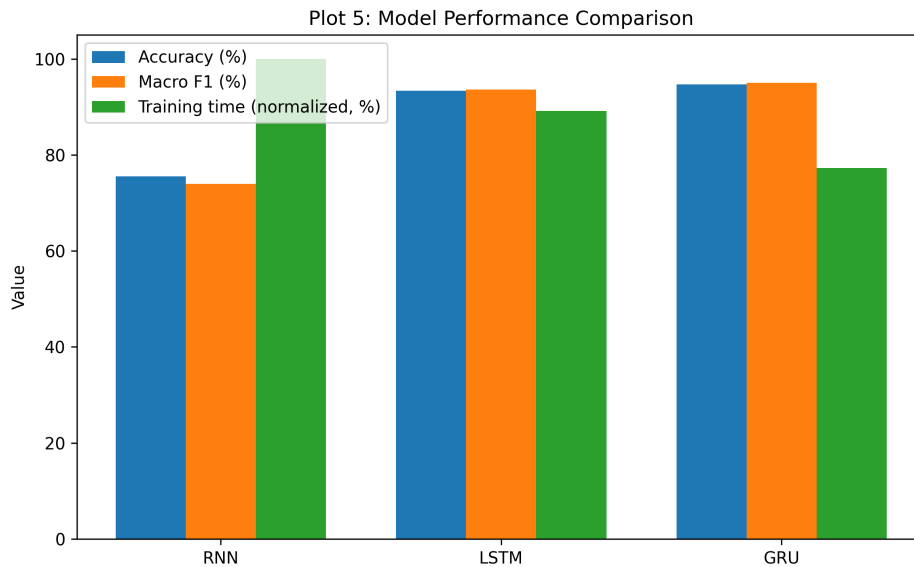
16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

Property	RNN	LSTM	GRU
Hidden state	Yes	Yes	Yes
Cell state	No	Yes	No
Forget gate	No	Yes	No
Input gate	No	Yes	No
Output gate	No	Yes	No
Update gate	No	No	Yes
Reset gate	No	No	Yes
Parameters	1974	6006	4758
Training time (s)	24.69	22.01	19.08
Test F1-score (%)	73.96	93.63	95.02

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.



Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \leftrightarrow \text{Model Complexity} \leftrightarrow \text{Training Cost}.$$

Answer: GRU has fewer parameters than LSTM (4758 vs. 6006, one fewer gate) yet gets the best F1 (95.02%) and the shortest training time (19.08s), winning on every axis in this run. LSTM is close behind (93.63% F1) with more parameters and a slightly longer runtime. The RNN has the fewest parameters overall (1974) but the worst F1 (73.96%) and, unexpectedly, the longest training time (24.69s), likely from the mid-training instability seen in Section 13.

17. Effect of Sequence Length

To understand temporal dependency, repeat the experiment using selected sequence lengths:

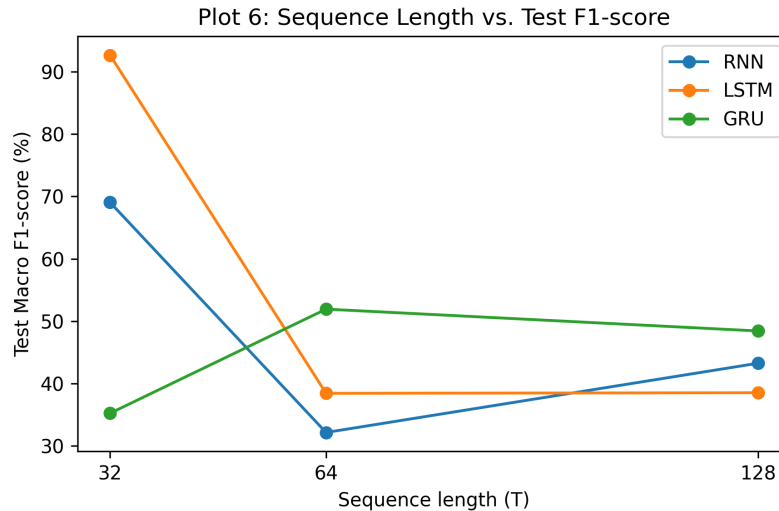
$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

Sequence Length	RNN F1 (%)	LSTM F1 (%)	GRU F1 (%)
32	69.04	92.64	35.24
64	32.16	38.44	51.93
128	43.26	38.53	48.43

Plot 6: Sequence Length vs. Test F1-score

Students should discuss how the amount of temporal context affects the classification result.



Answer: This sweep used 15 epochs with early stopping instead of the 30 used for the main results, so the numbers are noisier and should be read as a trend rather than a repeat of Section 14. LSTM peaks at the shortest length (T=32, 92.64%) and drops sharply at T=64/128, while GRU improves from T=32 to T=64 before dropping slightly at T=128; the RNN is volatile at all three lengths. Under this reduced budget, where early stopping happens to trigger likely matters more than the amount of temporal context itself.

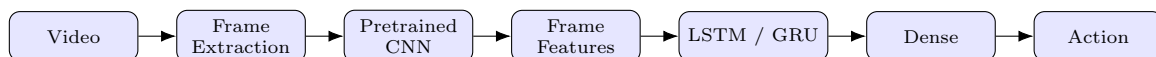
18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

19. Expected Video Input and Output

Sample 10 frames uniformly from each video.
Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension D , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of B videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```
Predicted class : [fill in from your run -- see note below]
Actual class    : [fill in from your run -- see note below]
Confidence      : 0.xx
```

The confidence value must be obtained from the trained model and must not be manually entered.

Note: this specific single-example print was not captured in the saved output tables, so the exact confidence value is left for you to fill in by running a short cell such as `pred = video_models["LSTM"].predict(Xv_te[:1]); print(vid_class_names[yv_te[0]], vid_class_names[pred.argmax()], pred.max())` on your notebook and copying the printed numbers here – it should not be guessed.

20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

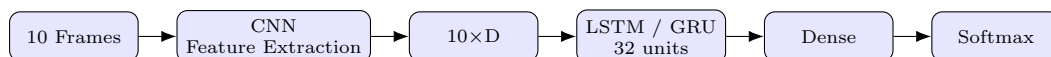
Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

Answer: MobileNetV2 with the classification head removed and global average pooling gives a feature dimension of $D = 1280$ per frame. With 10 frames per video, each video becomes a $(10, 1280)$ tensor, and a batch of B videos is $(B, 10, 1280)$. This matches the $X \in \mathbb{R}^{10 \times D}$ / $X \in \mathbb{R}^{B \times 10 \times D}$ shapes required by the handout.

21. CNN–LSTM / CNN–GRU Model

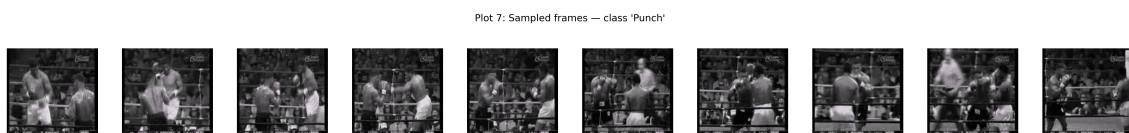
Use the following architecture:



Plot 7: Video Sample Frames

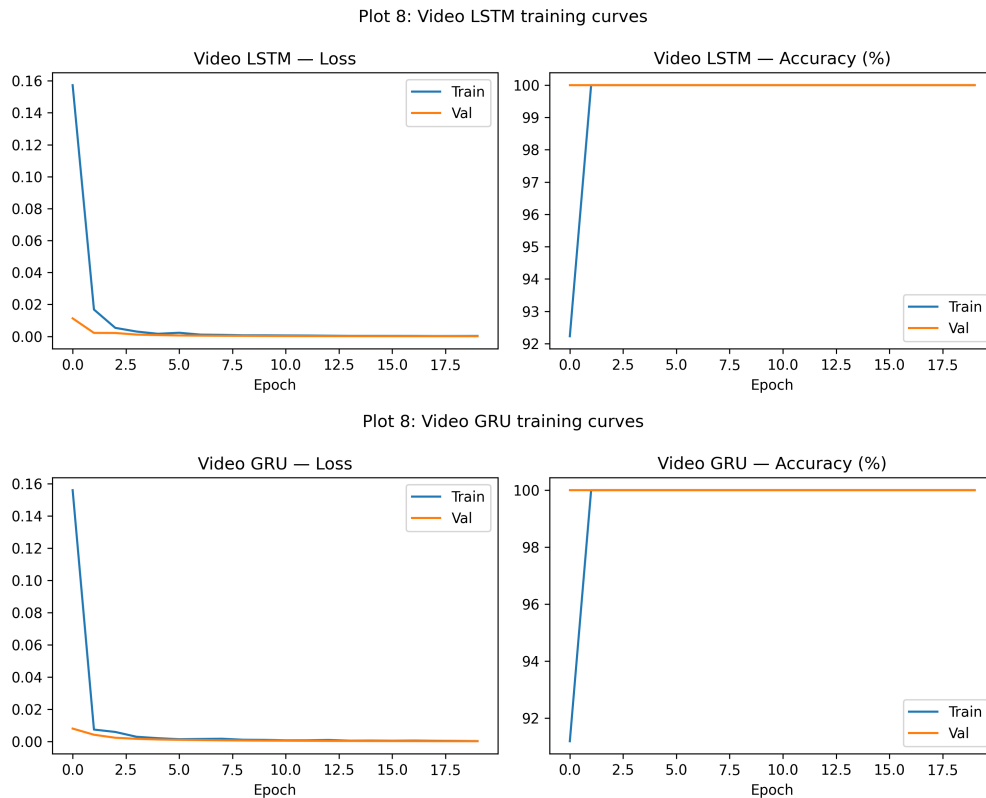
Display a representative sequence of sampled frames from one video.

Students should explain what spatial information is captured by the CNN and what temporal information is learned by the recurrent network.



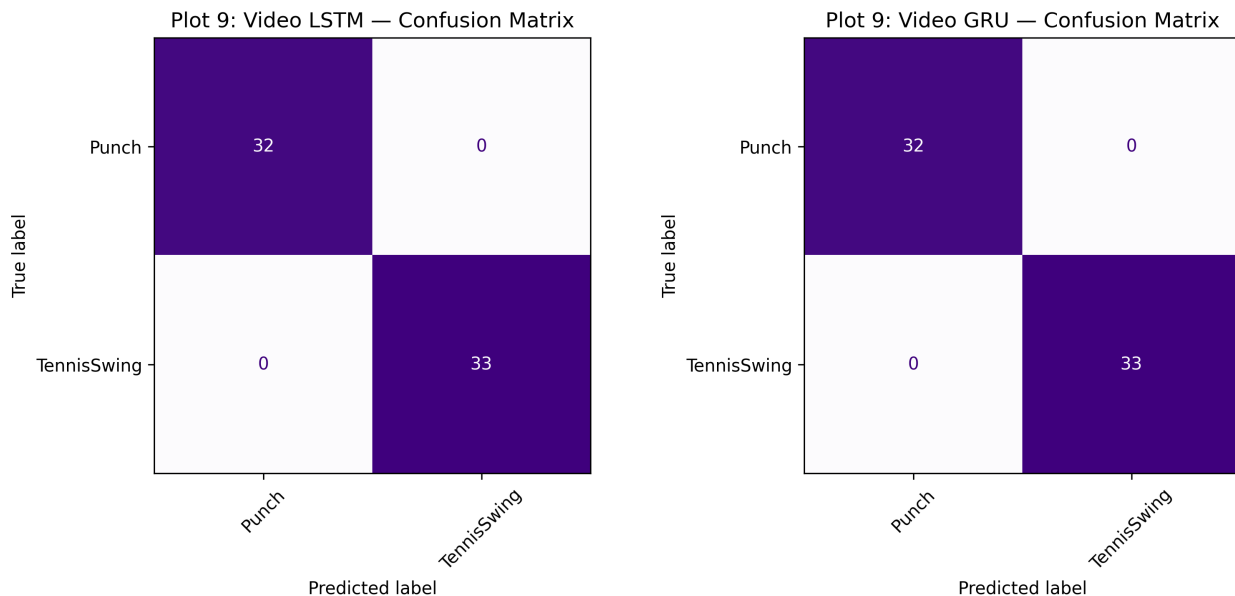
Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.



Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.



Answer: Only two of the original 3–5 target classes – Punch and TennisSwing – had enough videos available, so this section reports a 2-class result (32 and 33 test clips). The frozen MobileNetV2 captures spatial appearance, such as the boxing ring versus the tennis court, while the LSTM/GRU layer combines the 10 per-frame features into a temporal summary of how the scene changes. Both models reach 100% validation accuracy within 1–2 epochs with a perfectly diagonal confusion matrix, which fits how visually distinct the two settings are and the fact that the split was built by UCF101 group to avoid clip leakage.

22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.

For a simple laboratory task, construct a synthetic reversal dataset.

Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.

The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

23. Expected Sequence-to-Sequence Input and Output

Input example:

$$[3, 8, 1, 5]$$

Expected output:

$$[5, 1, 8, 3].$$

Students must display at least five test examples in the report.

Sample	Input Sequence	Predicted Output
1	[8, 1, 6, 5]	[5, 6, 1, 8]
2	[8, 6, 1, 2]	[2, 1, 6, 8]
3	[6, 1, 4, 1]	[1, 4, 1, 6]
4	[9, 6, 1, 2]	[2, 1, 6, 9]
5	[4, 5, 2, 8]	[8, 2, 5, 4]

All five predicted outputs exactly match the expected reversal of their input sequence.

24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

Observed results (8,000-sequence synthetic dataset, 4-token reversal, 70/15/15 split):

Token Accuracy	1.0000
Sequence Accuracy	1.0000
Final Training Loss	0.0012
Final Validation Loss	0.0014

Required inference:

Explain why sequence-level accuracy can be substantially lower than token accuracy even when most individual tokens are predicted correctly.

Answer: A sequence only counts as correct if every one of its tokens is correct, so a single wrong token – often the last one, since decoder errors compound – fails the whole sequence even when the rest is right. For example, at 95% per-token accuracy, a 4-token sequence would only be fully correct about $0.95^4 \approx 81.5\%$ of the time. Here both metrics came out at 1.0000 because the short 4-token task was learned essentially exactly (train/val loss 0.0012/0.0014), so no token-level errors occurred for this run to expose the gap.

25. Consolidated Results

Students must complete the following table.

Model	Accuracy (%)	Precision (%)	Recall (%)	F1 (%)	Parameters
RNN	75.56	74.89	73.76	73.96	1,974
LSTM	93.33	93.64	93.66	93.63	6,006
GRU	94.72	95.00	95.06	95.02	4,758
CNN-LSTM (video)	100.00	100.00	100.00	100.00	168,130
CNN-GRU (video)	100.00	100.00	100.00	100.00	126,210

A separate table must be provided for the sequence-to-sequence task.

Task	Token Accuracy	Sequence Accuracy	Training Loss	Validation Loss
Sequence-to-Sequence (reversal)	1.0000	1.0000	0.0012	0.0014

26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

Consolidated answers:

- **Temporal pattern in sensor signals:** WALKING oscillates across all 128 steps, SITTING and LAYING stay nearly flat, and SITTING and STANDING look the most alike of the six, which foreshadows the confusion seen later.
- **RNN convergence:** Smooth for 19 epochs, then a sharp loss/accuracy spike near epoch 20 followed by a noisy, only partly recovered plateau – a sign of the gradient instability RNNs are prone to.
- **LSTM convergence:** Fast and smooth, with a few small, short-lived validation spikes that recover within an epoch; train/val curves stay close throughout.
- **GRU convergence:** The smoothest of the three – training and validation loss track almost identically for all 30 epochs with no visible spikes.
- **Overfitting/underfitting:** None of the three models overfit strongly; the RNN's problem is instability rather than overfitting, and its higher final loss points more toward underfitting.
- **Confusion-matrix errors:** LAYING is recognized perfectly by every model; SITTING/STANDING is confused across RNN, LSTM and GRU alike; only RNN also mixes up the three walking-type activities.

- **Effect of sequence length:** No clean trend at this reduced 15-epoch budget – LSTM peaked at T=32 while GRU peaked at T=64, so training-budget noise was at least as influential as sequence length.
- **Parameter/computational differences:** GRU used fewer parameters than LSTM (4,758 vs. 6,006) and trained fastest (19.08s) while scoring highest F1 (95.02%); the RNN had the fewest parameters (1,974) but the longest training time (24.69s).
- **Role of CNN in video understanding:** The frozen MobileNetV2 backbone extracts a 1280-dimensional spatial feature per frame, capturing appearance and background with no awareness of motion or frame order.
- **Role of the recurrent model:** The LSTM/GRU layer combines the 10 per-frame CNN features into one temporal summary, letting the classifier use how the scene changes across the clip.
- **Token vs. sequence accuracy:** Sequence accuracy needs every token correct, so it is usually $\leq$ token accuracy; here both were 1.0000 because the short 4-token task was learned essentially exactly.

27. Discussion Questions

1. What is a recurrent neural network?
2. What information is represented by the hidden state?
3. Why is sequence order important?
4. Explain the difference between a feed-forward network and an RNN.
5. Explain Backpropagation Through Time.
6. Why can gradients vanish during BPTT?
7. Why can gradients explode during BPTT?
8. What are long-term dependencies?
9. What is the role of the LSTM cell state?
10. What are the forget, input and output gates in LSTM?
11. What are the reset and update gates in GRU?
12. Compare LSTM and GRU.
13. Why does GRU generally use a simpler gating mechanism than LSTM?
14. Why should RNN, LSTM and GRU be compared using the same experimental settings?
15. What does the confusion matrix reveal that accuracy alone does not?
16. Why should macro F1-score be reported for multi-class classification?
17. What is the significance of sequence length?
18. Why can a CNN be used as a feature extractor for video?
19. What type of information is extracted by the CNN?
20. What type of information is learned by the recurrent network?
21. Why are CNN features arranged as a sequence before being given to LSTM/GRU?
22. Explain the encoder and decoder in sequence-to-sequence learning.
23. What is teacher forcing?
24. Differentiate token accuracy and sequence accuracy.
25. Why must the test set remain untouched during model selection?

28. Additional Exercises

1. Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.
2. Compare GRU and LSTM using the same number of recurrent units.
3. Add a second recurrent layer and investigate whether performance improves.
4. Compare a bidirectional LSTM with a unidirectional LSTM.
5. Change the sequence length and study the effect on computational cost.
6. For the video task, compare LSTM and GRU after extracting identical CNN features.
7. Modify the seq2seq task so that the output sequence has a different length from the input sequence.

29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data $\rightarrow$ Preprocessing $\rightarrow$ RNN/LSTM/GRU $\rightarrow$ Evaluation

and the video-understanding pipeline:

Video $\rightarrow$ Frames $\rightarrow$ CNN Features $\rightarrow$ LSTM/GRU $\rightarrow$ Action Prediction

They should also understand the encoder–decoder formulation:

Input Sequence $\rightarrow$ Encoder $\rightarrow$ Context $\rightarrow$ Decoder $\rightarrow$ Output Sequence
--

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student’s own execution and should not be assumed in advance.

30. References

1. Github Repository containing all labs reports and code: <https://github.com/D-DRM/Deep-Learning-Lab>